

ATDRPS

Advanced Threat Detection and Reinforcement Protocol System

A world model for network attack forecasting

Every IDS asks

"Is this flow malicious?"

ATDRPS asks

"Where is this network heading?"

It learns $P(S_{t+1} | S_{\leq t})$ over network state and rolls it forward K steps — so a scan that is still only a scan can be read as a trajectory toward compromise.

Recon

>

Initial
Access

>

Lateral
Movement

>

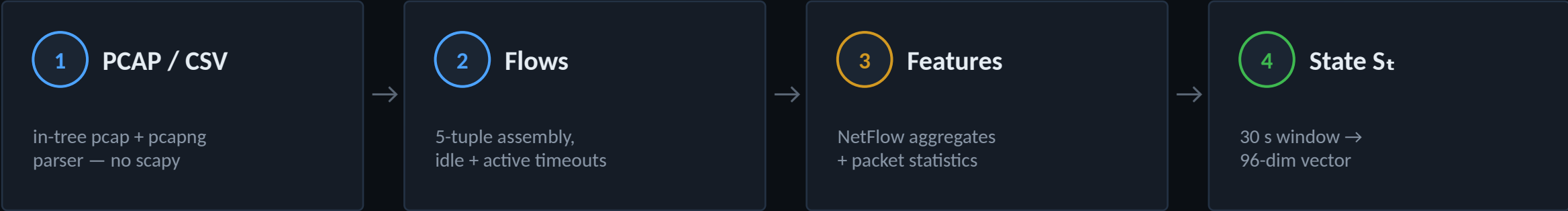
Command
& Control

>

Exfiltration

From packets to a network state

Dual-level features, because flow records erase exactly the evidence the hard cases need



What one window contains

- Volume and shape (22) — rates, distinct hosts and ports, direction, concentration
- Distributional aggregates (60) — mean, spread and peak of 20 per-flow features. Spread is the signal: beacons have near-zero timing variance

Behavioural detectors (14) — ports swept per pair, beacon regularity, internal peers reached, never-before-seen destinations

Measured separation

feature	benign	attack	
max ports per pair	1.3	22.8	Recon
admin service share	0.00	0.36	Initial Access
beacon regularity	0.66	0.92	C2
outbound byte ratio	0.01	0.52	Exfiltration

A slow scan sits under every flow-level threshold. What gives it away is constant TTL, a constant tiny window, zero payload and a walk across ports — all of which aggregation destroys.

The world model

One encoder, three heads, and a forward simulation

Temporal transformer

`L = 16 context windows · d_model 128 · 4 heads · 3 pre-norm blocks`



next-state Δ

how the state changes — the dynamics target



stage head

MITRE ATT&CK stage of the next window

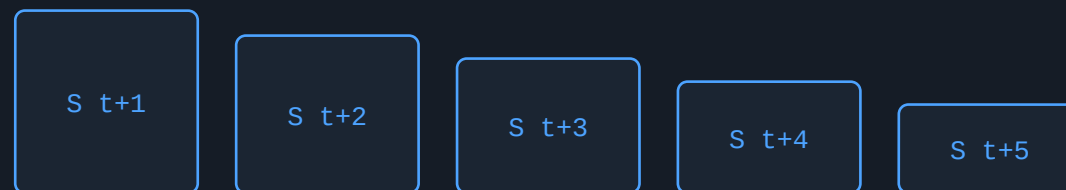


infiltration head

probability infiltration is underway next

K-step forward simulation

predict → append → re-predict. Errors compound, and they should — a model equally confident at step 5 as step 1 is lying about what it knows.



confidence decays with distance

Predicted states are clamped to the range training data covered, so a simulation cannot wander into states no network has ever produced.

Training all three together is the point. The stage head alone is a classifier with extra steps; making one representation also reconstruct the next state is what forces it to learn dynamics.

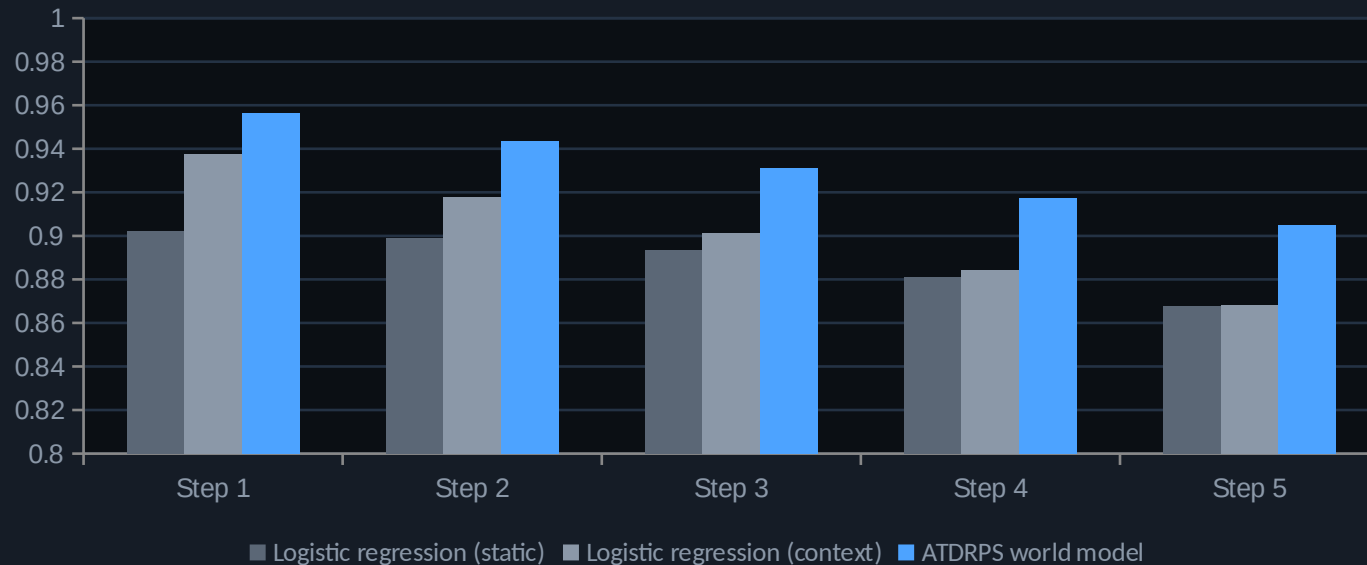
The fix that made it work: predict the change, not the state

Regressing the absolute next state looked fine in unit tests. Rendering the first live forecast exposed it: five steps read 1.00, 0.00, 0.00, 0.99, 1.00. Ridge shrinks toward zero, so a regularised model predicts the mean state — and feeding mean states back into the context made the heads swing between extremes. Predicting $S_{t+1} - S_t$ makes shrinkage mean "nothing changes", which is the right prior for network state. False-positive rate fell from 0.134 to 0.071.

Results

Held-out captures — train and test share no window, no flow, no campaign. Thresholds chosen on validation.

Infiltration forecast F1 by horizon step



False-positive rate

0.071

vs 0.112
logreg (context)

Is it really a world model?

Next-state error against the persistence floor. A model that wins on F1 but cannot beat "assume nothing changes" has learned a classifier, not dynamics.

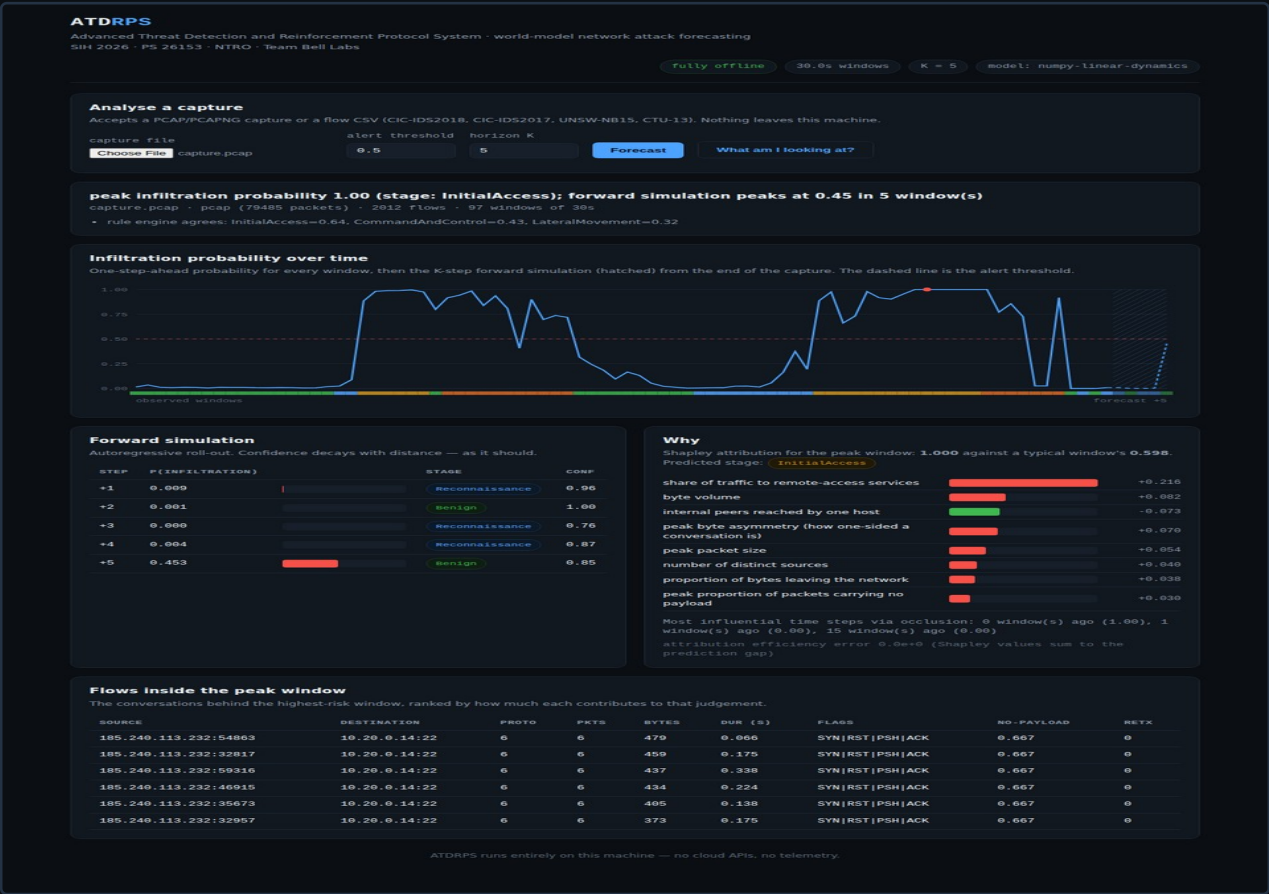


The margin widens the further ahead it forecasts — which is the whole claim

At one window ahead the world model beats the strongest logistic baseline by 0.019 F1. At five windows ahead the gap is 0.037, and the false-positive rate gap is 0.183 against 0.233. A static classifier has nothing to extrapolate from; a learned transition model does. Verified end to end on an unseen capture: 80% exact stage match and the Recon → Initial Access → Lateral Movement ordering recovered, including the slow scan.

Explainable, and it runs in an air-gapped network

Both are requirements, not features — and both shaped what we were willing to depend on



The offline dashboard: risk timeline, forward simulation, Shapley drivers and the flows behind the peak window.
79,485 packets to rendered result in 3.9 seconds.

What drove it

Grouped KernelSHAP over the 96 features — written in-tree, verified against closed-form Shapley values to 1.2e-05. Reported in plain language: "share of traffic to remote-access services +0.216".

When it came from

Attention across the 16 context windows, with model-agnostic occlusion as a cross-check — so every backend is explainable, not just the transformer.

Why no scapy, no shap

CII networks are air-gapped. ATDRPS ships its own pcap/pcapng parser and its own KernelSHAP, so there is one less supply chain to vet and patch. No cloud API, no telemetry, no runtime download.

215 tests · standard library only · MIT licensed ·
reproducible from (config, dataset, seed)